iAM

Intent becomes action.

MVP Product Overview

4-Week Sprint · All 7 Services · Stage Profiles · Gift Links

Name	Role	Location
Maaz	CTO	Pakistan
Gilson	GTM	Curaçao
Nekha	Founding Engineer	Bay Area, US
Ashish	Founding Designer	India
Samantha	Lead Designer	Australia

1. Executive Summary

iAM is a collaborative intent-to-action platform. Users type a plan in natural language, tag friends, and get a unified workspace — the Stage — where all relevant services are assembled instantly, locked collaboratively, and acted on in one place.

The MVP ships in 4 weeks. Basic intent parsing, flight search, and a checkout skeleton already exist — those get finished, not rebuilt. The sprint focuses on completing all seven integrated services, launching User and Brand Stage profiles, building the Gift Link flow, and proving the product with real users and real bookings.

4-week goal:

20 real users onboarded across trip planning and gift link scenarios. 3 to 4 completed bookings with real money. Full flow from prompt to confirmation working across all 7 services.

Feature	Status
Intent parsing + NLP layer	In progress — polish only
Duffel flights search + booking	In progress — finish
Duffel Rental Car	To Build
Basic checkout skeleton	In progress — complete
Duffel stays (hotels)	In progress — complete
Viator experiences	To Build
OpenTable restaurants	To Build
OpenWeatherMap	In progress — polish only
Google Maps POIs + shopping	To Build — lightweight
Collaborative Stage + real-time sync	In progress — finish
User & Brand Stage profiles	To Build
Gift Link flow	To Build
iAM learning + intent graph	To Build

2. Full Feature Scope

2.1 Seven integrated services

The Stage builds instantly by calling all seven services in parallel. Each service populates its own row. Rows stream progressively as each call resolves — no blank wait screen.

Service	Provider	What it does in the Stage
Flights	Duffel	Real-time routes, pricing, seat options — bookable inside the Stage
Hotels & Stays	Duffel	Accommodations filtered by group size, dates, budget signal — bookable
Rental Car	Duffel	Rental Car booking filtered by comfort and number of passengers
Activities & Tours	Viator	Museum tickets, river cruises, guided tours — bookable
Restaurant Reservations	OpenTable	Table availability matched to group size, date, cuisine preferences — reservable
Weather Forecast	OpenWeatherMap	Local forecast for trip dates — informs packing suggestions and clothing cards
Maps & Shopping POIs	Google Maps	Nearby malls, markets, key shopping areas — displayed as a discovery row

Booking scope note:

Flights, stays, activities, and restaurant reservations are all bookable inside the Stage. Weather and Maps are informational rows — no transaction, no checkout action. They enrich the Stage without adding checkout complexity.

2.2 User and Brand Stage profiles

Every iAM user and brand gets their own Stage — a profile page that functions as both a public presence and a live intent interface.

What a Stage profile looks like

The layout takes cues from Instagram and LinkedIn: a clean header with profile photo, display name, @handle, short bio, and location. Below the header, a prompt input sits permanently at the bottom of the profile. Visitors can type questions or requests and get answers powered by the iAM Stage engine — text only for MVP.

Element	Details
Header image	Full-width cover image — uploaded by user or brand
Avatar	Profile photo — circular, prominent

Display name	Full name or brand name
@handle	Unique identifier — used for tagging in Stage prompts
Short bio	Max 160 characters — plain text for MVP
Location	City or region — optional
Prompt input	Text field at the bottom of the profile — visitors ask questions, iAM answers using the profile owner's intent graph and uploaded document
Post-MVP additions	Photos, music, shared items, product wishlists, collaborative trip boards

Document upload and profile seeding

Users and brands upload a document to seed their Stage profile. iAM provides a sample document template that guides what to include: interests, preferences, style, favourite Items, dietary preferences, brand values (for brand accounts).

- iAM provides a downloadable sample document (PDF and DOCX formats) at onboarding.
- User fills in their details and uploads — iAM parses the document and populates the intent graph.
- The intent graph drives card ranking, Stage personalization, and responses to visitors on the profile prompt.
- Document stored in Blob — parsed content stored in MongoDB intentGraphs collection.
- Users can re-upload at any time to update their profile and preferences.

Profile prompt — how it works

When a visitor types into the profile prompt, iAM treats it as an intent query filtered through the profile owner's context. Example: visiting a travel blogger's Stage profile and typing "what should I pack for Tokyo in March" produces a personalised answer shaped by that blogger's documented preferences, past trips, and style signals.

For MVP, responses are text-only. The prompt is not a chatbot — it is a Stage query narrowed to the profile owner's world. Post-MVP, responses surface visual cards, product recommendations, and bookable items.

2.3 Gift Link

The Gift Link feature lets any iAM user buy an item for someone else — without needing the recipient's address upfront. The sender handles everything except the delivery address. The recipient provides only that.

How Gift Link works — sender flow

1. Sender searches for an item inside any Stage — a product, experience, hotel stay, or activity
2. Sender taps Buy as Gift on any bookable card
3. A gift checkout form opens: sender fills in all booking details (dates, specifications, quantity, payment) but leaves the delivery address blank
4. Sender's card is saved against the gift order — payment is not taken yet
5. iAM generates a unique gift link: iam.app/gift/{token}
6. If the recipient has an iAM account, they receive an in-app notification directly
7. If not on iAM, the sender copies or shares the link via any channel — message, email, WhatsApp

How Gift Link works — recipient flow

8. Recipient opens the gift link or taps the in-app notification
9. They see a gift reveal page: what was bought, from whom, and a short personal message if the sender included one
10. Recipient enters only their delivery address — nothing else required
11. On submission, payment is deducted from the sender's saved card
12. Order is created with the vendor — shipped or confirmed to the recipient's address
13. Both sender and recipient receive a confirmation — sender sees fulfilment, recipient sees booking details

Gift Order field	Details
id	Unique gift order identifier
token	URL-safe random token — used in the gift link
fromUserId	Sender's iAM user ID
toUserId	Recipient's iAM user ID — null if not on iAM
toEmail / toPhone	Used for notification if recipient is not on iAM
item	Full item details — vendor, product ID, specs, dates, amount
paymentMethodId	Stripe payment method saved from sender — not charged until address provided

personalMessage	Optional short note from sender to recipient
status	pending_address, address_received, payment_processing, confirmed, failed
shippingAddress	Null until recipient provides it
createdAt / redeemedAt	Timestamps for both ends of the flow

Key principle:

The sender pays. The recipient only gives an address. No awkward money conversations. No exposing the recipient's address to the sender — iAM holds it and routes it directly to the vendor.

2.4 iAM learns — intent graph and personalization

After every interaction — a booking completed, an item locked, a Stage built — iAM updates the user's intent graph. This makes every subsequent Stage more accurate and more personal.

- **Destinations:** tracks where users have been, searched for, and booked
- **Spending signals:** budget, mid-range, or premium — inferred from booking choices, never asked directly
- **Activity preferences:** museums, outdoor, nightlife, food-led — built from Viator and OpenTable engagement
- **Travel style:** solo, couples, group — inferred from participant counts across bookings
- **Seasonal patterns:** when users tend to travel and to where

This data is used exclusively to improve the Stage experience for that user. It is never sold, never shared with advertisers, and never used to build targeting profiles for external parties. The intent graph belongs to the user.

3. User Workflows

3.1 Scenario A — Group trip planning

Maaz types: "Plan a 3-day trip to Paris with @Gilson and @Sam — flights from London, mid-range hotel, need a car, show me what to do and where to eat."

14. iAM parses the intent — destination: Paris, origin: London, duration: 3 days, participants: @Gilson and @Sam, budget: mid-range

15. @Gilson and @Sam receive SSE notifications and join the Stage instantly

16. All seven services fire in parallel — flights, stays, Cars, Viator experiences, OpenTable restaurants, weather forecast, and Google Maps shopping areas

17. Stage rows stream in progressively. Maaz, Gilson, and Sam all see the same Stage in real time

18. Maaz locks a flight. Gilson locks a hotel room. Sam locks a Viator boat tour and an OpenTable dinner reservation

19. Maaz taps Checkout — combined cart shows flight + hotel + tour + dinner. One Stripe payment

20. Payment clears — vendor split routes correct amounts to Duffel (flight and hotel), Viator, and OpenTable

21. All three receive a combined confirmation email and in-Stage confirmation card

22. Intent graphs updated for Maaz, Gilson, and Sam — Paris, mid-range, group of 3, late spring

3.2 Scenario B — Brand Stage profile

A boutique Paris hotel creates a brand Stage profile. They upload their brand document — hotel philosophy, room types, typical guest profile, nearby attractions.

23. Hotel fills in the iAM sample document — brand values, property highlights, target guest description

24. They upload it to their Stage profile — iAM parses and populates the intent graph for this brand

25. Their Stage profile is live at iam.app/@lemaishotel — header image, logo, bio, location

26. A user visits the profile and types: "Is this hotel good for a solo business trip in June?"

27. iAM answers using the hotel's intent graph context — room types, amenities, past booking patterns — text response for MVP

3.3 Scenario C — Gift Link

Maaz wants to buy a Louvre skip-the-line ticket for his friend Sam who is visiting Paris next month.

28. Maaz opens a Stage, searches for Louvre experiences via Viator

29. He finds the right ticket — taps Buy as Gift

30. Gift checkout opens — Maaz selects date, quantity (1), adds a personal message, enters his card
31. He does not enter a delivery address — that field is locked for the recipient
32. iAM generates the gift link — iam.app/gift/{token}
33. Sam has an iAM account — she receives an in-app notification: "Maaz sent you a gift!"
34. Sam opens the gift reveal page, sees the Louvre ticket details and Maaz's message
35. Sam enters her address (or hotel address for the booking confirmation) and confirms
36. Payment is charged to Maaz's card — Viator booking created under Sam's name, confirmation sent to both

4. Technical Data Flow

4.1 Intent to Stage assembly

#	Step	Module	Output
1	User submits prompt	app/api/intent/route.ts	API request received
2	Parser extracts intent fields	lib/intent/parser.ts	ParsedIntent object
3	@handles resolved from DB	lib/intent/participants.ts	Participant user IDs
4	SSE notifications dispatched	lib/sse/notify.ts	Participants join Stage
5	Intent graphs merged for all participants	lib/stage/merge.ts	Merged StageContext
6	Promise.all() fires across 7 services	lib/stage/assembler.ts	Parallel API calls
7	Each response passes relevance gate	lib/ranking/gate.ts	Filtered card set
8	Cards ranked — Intent Fit + User Fit + Outcome History	lib/ranking/ranker.ts	Ranked rows
9	Rows streamed progressively to UI	app/api/stage/stream/route.ts	Stage visible
10	Stage state cached in Upstash Redis	lib/cache/stageState.ts	Live state persisted

4.2 Service integration details

Service	Type	Key API action	Response cached?

Duffel flights	Transactional	Offer search, order creation, booking confirmation	Yes — 15 min TTL
Duffel stays	Transactional	Hotel search, room offers, booking	Yes — 15 min TTL
Duffel Renta Car	Transactional	Search Available Cars, category filtes, booking	Yes — 15 mints TTL
Viator experiences	Transactional	Product search, availability check, booking	Yes — 30 min TTL
OpenTable	Transactional	Availability search, reservation creation	Yes — 5 min TTL
OpenWeatherMap	Informational	Current + forecast for destination and dates	Yes — 1 hr TTL
Google Maps	Informational	Nearby search for POIs, shopping, landmarks	Yes — 6 hr TTL

4.3 Locking and cart

#	Step	Module	Data stored
1	User taps lock on a card	app/api/stage/lock/route.ts	Lock request received
2	CartItem created	lib/checkout/types.ts	cardId, vendorId, amount, lockedBy, isShared
3	Cart updated in Zustand	stores/cartStore.ts	Client state reflects lock
4	Lock broadcast to all participants	lib/sse/broadcast.ts	All participants see lock
5	StageCart persisted to MongoDB	lib/db/mongo.ts	stageCarts collection updated

4.4 Checkout and vendor split

#	Step	Module	Notes
1	Initiator taps Checkout	app/api/checkout/route.ts	Cart validated — all items locked
2	PendingOrder created	lib/checkout/pendingOrder.ts	Offer IDs, total, expiry stored
3	Stripe PaymentIntent created	lib/payments/stripe.ts	Combined total, single charge

4	User completes Stripe checkout	Stripe hosted elements	Payment submitted
5	Stripe webhook fires	app/api/webhooks/stripe/route.ts	payment_intent.succeeded
6	Idempotency check	lib/checkout/split.ts	processedSplits collection
7	Stripe Connect transfers per vendor	lib/checkout/split.ts	Duffel, Viator, OpenTable paid
8	Duffel order created — iAM pays from balance	lib/services/duffel/order.ts	Flight + hotel booked
9	Viator booking confirmed	lib/services/viator/order.ts	Experience confirmed
10	OpenTable reservation confirmed	lib/services/opentable/order.ts	Table reserved
11	Confirmation email sent	lib/mail.ts	Resend — all participants
12	In-Stage confirmation card shown	components/Stage/Confirmation.tsx	SSE push to all
13	Intent graph updated	lib/intent/graph.ts	Outcome signals stored

4.5 Gift Link data flow

#	Step	Module	Notes
1	Sender taps Buy as Gift	app/api/gifts/create/route.ts	Gift checkout opened
2	GiftOrder created — status: pending_address	lib/gifts/giftOrder.ts	Token generated, card saved
3	If recipient on iAM: in-app notification	lib/sse/notify.ts	Push to recipient session
4	If not on iAM: gift link generated	iam.app/gift/{token}	Sender shares link
5	Recipient opens gift reveal page	app/gift/[token]/page.tsx	Item details displayed
6	Recipient submits address — status: address_received	app/api/gifts/redeem/route.ts	Address stored
7	Stripe charge fires against sender's saved card	lib/payments/stripe.ts	Payment taken now
8	Vendor order created with recipient address	lib/services/*/order.ts	Booking confirmed
9	Confirmation sent to both parties	lib/mail.ts	Sender sees fulfilment, recipient sees details

4.6 Stage profile data flow

#	Step	Module	Notes
1	User uploads document	app/api/profile/upload/route.ts	Stored in Vercel Blob
2	Document parsed — interests, preferences extracted	lib/profile/docParser.ts	Fields mapped to intent graph
3	StageProfile created or updated	lib/db/mongo.ts	stageProfiles collection
4	Profile page live at iam.app/@handle	app/[handle]/page.tsx	Header, bio, prompt visible
5	Visitor types into profile prompt	app/api/profile/prompt/route.ts	Query received
6	Query answered using profile owner's intent graph	lib/intent/profileQuery.ts	Text response returned
7	Response displayed below prompt input	components/Profile/PromptResponse.tsx	Text only — MVP

4.7 MongoDB collections

Collection	Key fields	Purpose
users	_id, email, handle, intentGraph, createdAt	User accounts
stageProfiles	userId, type, headerImage, avatar, bio, handle, uploadedDocUrl	User and brand profile pages
intentGraphs	userId, destinations[], preferences{}, outcomeHistory[], spendingSignal	Personalization data per user
stages	_id, initiatorId, participants[], status, createdAt	Active and historical Stages
stageCarts	stageId, participants[], items[], status	Live cart state per Stage
pendingOrders	_id, stageId, vendorItems[], totalAmount, payerId, status, expiresAt	Pre-payment holding state

processedSplits	paymentIntentId, processedAt	Idempotency guard
orders	_id, stageId, vendorOrders[], participants[], bookedAt	Confirmed booking records
giftOrders	_id, token, fromUserId, toUserId, item{}, paymentMethodId, status, shippingAddress	Gift link orders

5. MVP Success Metrics

Metric	Target	Why it matters
Real users onboarded	20	Enough for qualitative signal across 3 use cases
Completed group bookings	3 to 4	Real money — core investor traction signal
Gift links created and redeemed	2 to 3	Proves the gifting flow end-to-end
Stage profiles created	5 to 10 (including 1 to 2 brands)	Validates profile use case
Average trip booking value	> $1,200	Revenue thesis validation
Stage assembly time (all 7 services)	< 10 seconds	Speed is part of the magic
Booking confirmation success rate	100%	Zero dropped orders — table stakes
Profile prompt response quality	Positive qualitative feedback from beta users	Personalization thesis validation

6. Risks and Mitigations

Risk	Impact	Mitigation
Viator or OpenTable API access not approved in time	Those rows missing at launch	Gilson initiates partner contacts in Week 1. If delayed, those rows show a coming soon state — does not block booking flow for flights and stays.

4-week timeline too compressed for 7 services	Incomplete or unstable product	Flights + stays + Cars +checkout are the hard floor. Viator and OpenTable are additive. If they slip to Week 5, core thesis is still provable.
Gift link payment held indefinitely if recipient never redeems	Funds locked on sender card	Expiry on gift orders — 3 days. After expiry, gift automatically cancelled and no charge taken.
Document parser produces low-quality intent graph	Profile prompt responses poor	Start with structured sample document that maps directly to known intent fields. Unstructured parsing is post-MVP.
Beta users do not complete real bookings	Traction signal insufficient	Gilson concierges' bookings — actively facilitates. Target users with trips already planned, not hypothetical.
SSE drops during multi-participant Stage session	Participants see inconsistent state	Client-side reconnection with backoff. Stage state rehydrated from Redis on reconnect. No data loss.

iAM

Intent becomes action.

4 weeks. 7 services. Real bookings.

The north star:

iAM will never show you something only because someone paid. Payment can shift position. It cannot create relevance. Everything else can evolve — this does not.